

Lab - Pushing an Image to Azure Container Registry (ACR)

To Begin with the Lab

Summary

This lab demonstrates how to push a locally built Docker image to Azure Container Registry (ACR). The process includes enabling the admin user, logging in to ACR, tagging the local image with the registry name, and pushing it to the registry so it can be pulled and run from other Azure resources.

Understanding

Azure Container Registry (ACR) is a private Docker image registry in Azure. Before pushing an image, Docker must authenticate with the registry. The local image is tagged with the ACR login server name so Docker knows the destination registry. Once pushed, the image is stored centrally and can be pulled by Azure VMs, Azure Container Instances, AKS, or other services.

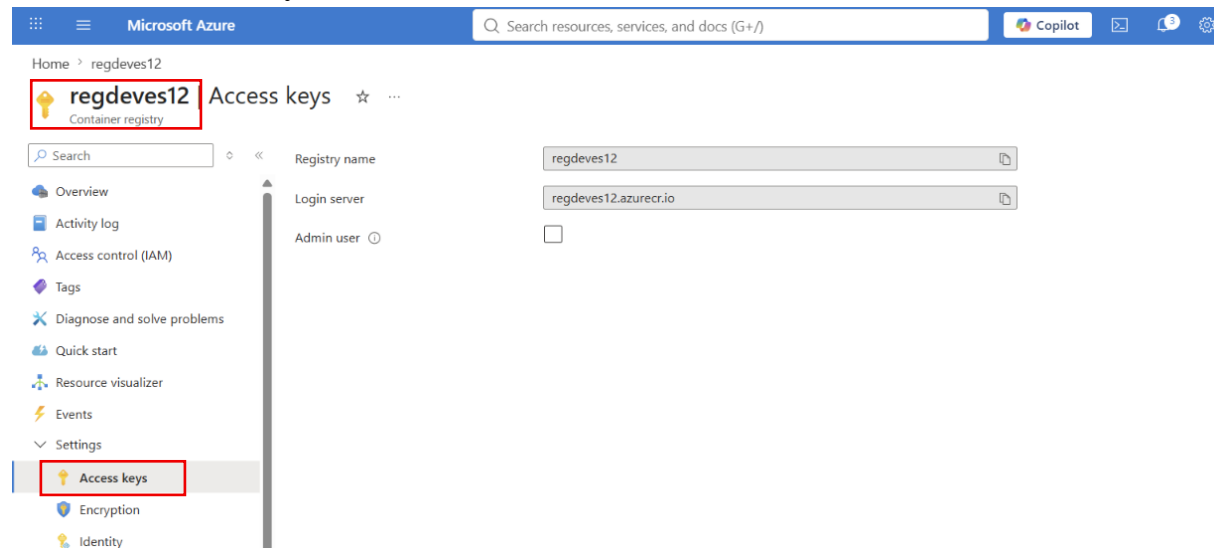
Example:

- Local Image: dockerapp:v1
- ACR Login Server: myregistry.azurecr.io
- Tagged Image: myregistry.azurecr.io/dockerapp:v1

Lab Steps

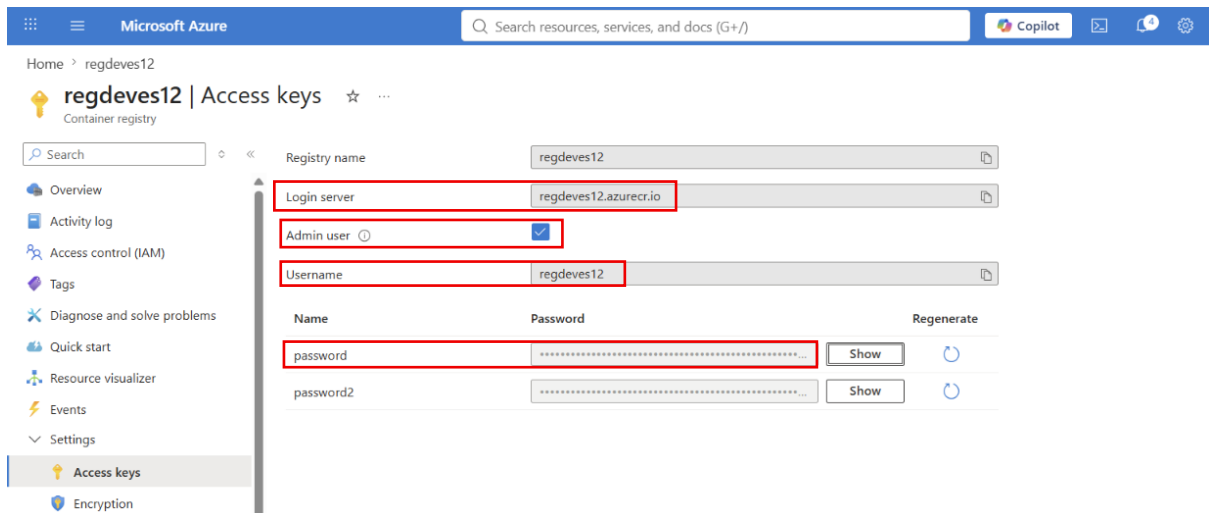
Step 1: Open Azure Container Registry

- Open the Azure Portal.
- Navigate to your **Azure Container Registry**.
- Select **Access Keys**.



Step 2: Enable Admin User

- Turn **Admin User** to **Enabled**.
- Copy the **Login Server**.
- Copy the **Username**.
- Copy either **Password** or **Password2**.

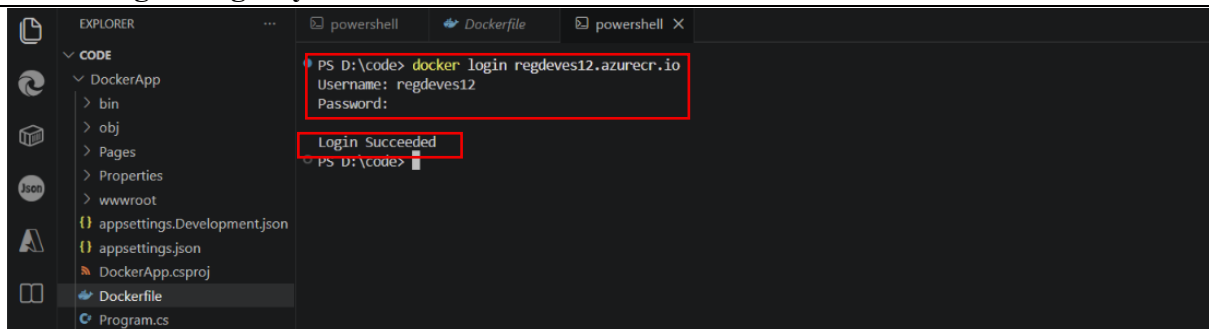


Note: In production environments, use **Azure RBAC** instead of the admin account for better security.

Step 3: Login to Azure Container Registry

- Use the ACR login server.
- Enter the **username**.
- Enter the **password**.
- **Login Succeeded.**

```
docker login <registry-name>.azurecr.io
```



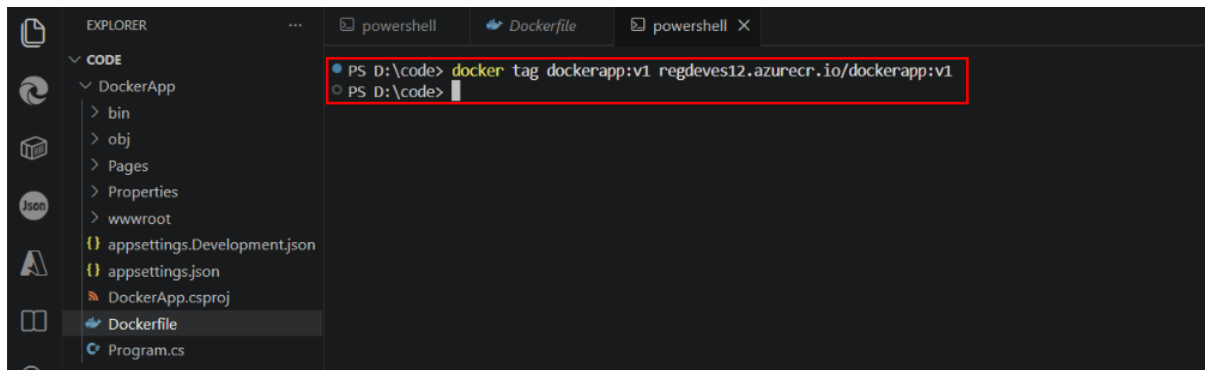
Step 4: Tag the Local Docker Image

Tag the local image with the ACR login server.

```
docker tag dockerapp:v1 <registry-name>.azurecr.io/dockerapp:v1
```

Why is tagging required?

- Docker needs to know the destination registry.
- Without the registry name, Docker assumes the image belongs to Docker Hub.
- Tagging tells Docker to push the image to Azure Container Registry.



```
PS D:\code> docker tag dockerapp:v1 regdeves12.azurecr.io/dockerapp:v1
PS D:\code>
```

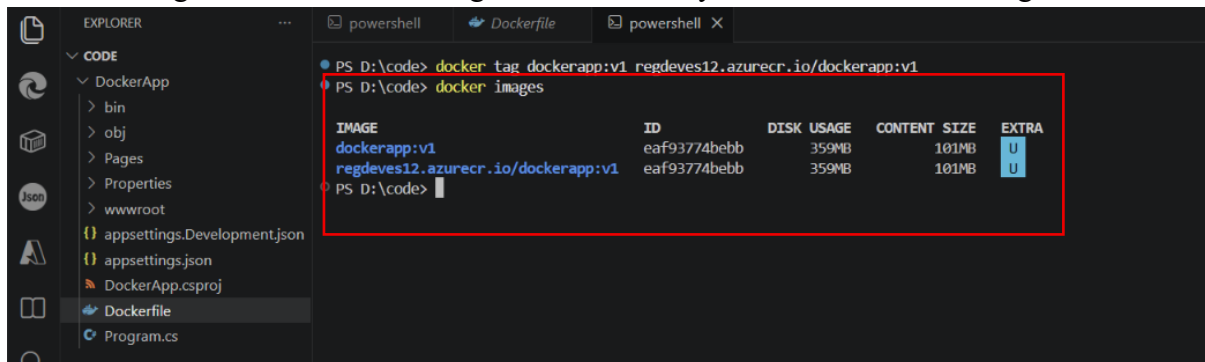
Step 5: Verify the Tagged Image

List the available Docker images.

```
docker images
```

Observe:

- The original image still exists.
- A new image appears with the ACR login server name.
- Both images share the same Image ID because they reference the same image.



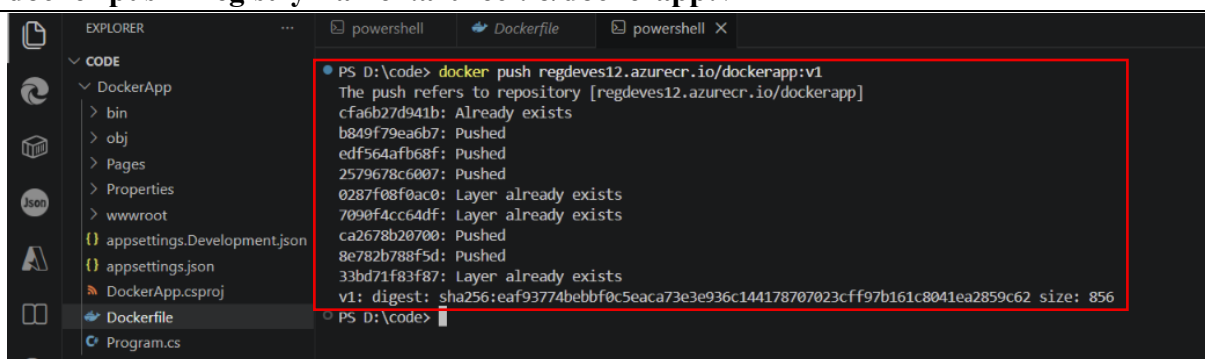
```
PS D:\code> docker tag dockerapp:v1 regdeves12.azurecr.io/dockerapp:v1
PS D:\code> docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
dockerapp:v1	eaf93774bebb	359MB	101MB	U
regdeves12.azurecr.io/dockerapp:v1	eaf93774bebb	359MB	101MB	U

Step 6: Push the Image to Azure Container Registry

Push the tagged image.

```
docker push <registry-name>.azurecr.io/dockerapp:v1
```



```
PS D:\code> docker push regdeves12.azurecr.io/dockerapp:v1
The push refers to repository [regdeves12.azurecr.io/dockerapp]
cfa6b27d941b: Already exists
b849f79ea6b7: Pushed
edf564afb68f: Pushed
2579678c6007: Pushed
0287f08f0ac0: Layer already exists
7090f4acc64df: Layer already exists
ca2678b20700: Pushed
8e782b788f5d: Pushed
33bd71f83f87: Layer already exists
v1: digest: sha256:eaf93774bebbf0c5eaca73e3e936c144178707023cff97b161c8041ea2859c62 size: 856
PS D:\code>
```

Step 7: Verify in Azure Portal

- Open Azure Container Registry.
- Select **Repositories**.
- Confirm that the repository appears.
- Verify the image tag (v1) is listed.

[Refresh](#) ...

Overview

Activity log

Access control (IAM)

Tags

Diagnose and solve problems

Quick start

Resource visualizer

Events

Settings

Services

Repositories

Webhooks

Search to filter repositories ...

Repositories ↑↓

dockerapp

...

dockerapp ...

Repository

[Refresh](#) [Start artifact streaming](#) [Manage deleted artifacts](#) [Delete repository](#)

^ Essentials

Repository	Tag count
dockerapp	1
Last updated date	Manifest count
7/21/2026, 12:00 PM GMT+5:30	3

Tags ↑↓	Digest ↑↓	Last modified	
v1	sha256:eaf93774bebbf0c5eaca73e3e936c14417...	7/21/2026, 12:00 PM GMT+5:30	...